

Estructura de Computadores Grado en Ingeniería Informática Curso 2025/2026

Proyecto de programación en ensamblador

Aclaraciones-Enunciado

Se incluirán en este apartado aclaraciones o respuestas proporcionadas a preguntas de alumnos que se considere puedan ser de interés general.

Uso de las entregas opcionales

- En cada una de las correcciones realizadas al código entregado por cada grupo se evalúa el funcionamiento de todas las subrutinas, independientemente de que se trate de la primera o la última corrección, la correspondiente al hito o cualquiera de las correcciones opcionales.

En consecuencia, recomendamos hacer un uso eficaz de las correcciones opcionales. Siempre que sea posible, es conveniente aprovechar cada corrección para comprobar el funcionamiento de alguna subrutina que se haya implementado desde la corrección anterior y no solo para ver si se ha solucionado algún error previamente detectado. En este sentido, dado que entre la primera corrección y la correspondiente al hito transcurren quince días entre los que hay dos correcciones opcionales, recomendamos que se corrijan cuanto antes los errores detectados en esa primera corrección y se hagan las pruebas correspondientes con el emulador, pero que se evite hacer inmediatamente una nueva entrega (opcional). Por el contrario, la recomendación es que se siga avanzando en el desarrollo de otras subrutinas, para que al emplear esas correcciones opcionales se obtenga ya una realimentación de su comportamiento que facilite así el avance en el desarrollo de todo el código.

El mismo criterio se debe aplicar posteriormente, durante el periodo comprendido entre el hito y la corrección definitiva.

Uso de direcciones de memoria no permitidas

- Según se especifica en el enunciado, no está permitido utilizar por los grupos de proyecto las direcciones de memoria `0x00000000` a `0x0000000F` ni las superiores a la `0x00010000`. El uso de estas direcciones dará lugar a que en ciertas ocasiones no se permita completar la entrega del código. En otros casos, el sistema de corrección automática puede detectar errores de funcionamiento a pesar de que el código evaluado aparente ser correcto.

Gestión de errores y/o casos límite

- No es necesario realizar más tratamiento de errores en el desarrollo del proyecto que el indicado en el enunciado de algunas de las subrutinas.
Por ejemplo, en el caso de la subrutina BuscaCar no se pide comprobar que el parámetro "C" corresponda efectivamente a un carácter ascii (un valor entre 1 y 255). Tampoco se pide verificar que "from" sea menor que "to" o que no haya un cero en "ref" que esté antes de "from". Por ello, no es necesario (ni se considerará favorable de cara a la calificación del proyecto) incorporar código para evitar el tratamiento de estos u otros casos erróneos, puesto que no se ha indicado en la especificación ni se contempla utilizarlos en ninguna de las pruebas.

ERROR: alineamiento a palabra erroneo

- Este tipo de error se produce porque se intenta leer o escribir una palabra en una dirección de memoria que **no** está alineada a palabra, es decir, que no es múltiplo de 4. Una palabra solo se puede leer o escribir sin error de las direcciones que sean múltiplos de 4: 0, 4, 8, 12, 16, Del mismo modo, media palabra solo se puede leer o escribir sin error de las direcciones pares: 0, 2, 4, 6, 8, 10, ... Sin embargo, un byte sí se puede leer o escribir en cualquier dirección de memoria.

Para leer de memoria una palabra (o media palabra) usando una dirección no alineada (o que se desconoce si está alineada o no), se debe hacer byte a byte en direcciones sucesivas y reconstruyendo la palabra en un registro del procesador. Del mismo modo, para escribir una palabra en una dirección de memoria no alineada, se debe separar previamente en bytes y escribirlos de uno en uno en direcciones sucesivas. En cualquier caso, ya sea una lectura o una escritura, se debe respetar el convenio "little endian".

Formato del texto comprimido (resumen)

En la siguiente descripción resumida, se ha tratado de simplificar en lo posible la descripción de los formatos utilizados en el proyecto: texto convencional (sin comprimir) y texto comprimido. La referencia completa sigue siendo el enunciado del proyecto y se complementa con el ejemplo detallado en la presentación del proyecto.

- El texto convencional, sin comprimir, consta de una secuencia de caracteres, almacenados como bytes sin signo, que finalizan con un NUL (byte con valor 0).
- En el proceso de compresión se recorren los bytes del texto convencional, desde el primero al último, copiando en la estructura que constituirá el texto comprimido una secuencia de elementos. Cada uno de estos elementos podrá ser de dos tipos: a) un carácter (el siguiente del texto original), o b) una estructura formada por tres bytes, que representan b1) el desplazamiento desde el comienzo del texto original donde se encuentra una tira de N caracteres coincidentes con los N que comienzan en la posición del siguiente carácter del texto original, y b2) el valor de N (la longitud de la subcadena coincidente). Cada elemento de esta secuencia llevará asociado un bit en el denominado "mapa de bits": tendrá valor 0 si lo que se ha copiado es un carácter individual, y tendrá valor 1 si lo que se ha copiado es una estructura de 3 bytes que describe una subcadena de 4 ó más caracteres coincidentes.
- La estructura completa que almacena un texto comprimido en el formato del proyecto está formada por tres partes: 1.-Cabecera, 2.-Mapa_de_bits y 3.-Car_y_Ref, donde "Car_y_Ref" es la zona en que se almacenan "caracteres" y "referencias a subcadenas previas". El contenido de cada una de estas partes es el que describe el siguiente resumen:
 1. Cabecera: está formada siempre por 5 bytes, cuyo significado está detallado en el enunciado: los dos primeros determinan la longitud del texto original, el siguiente es una constante (siempre un 1) y los dos últimos hacen referencia a la posición en que comienza la zona 3 (Car_y_Ref).

2. Mapa de bits: contiene uno o varios bytes, cuyos bits indican sucesivamente si el elemento que se encuentra a continuación en la zona 3-Car_y_Ref es un carácter o una referencia a una subcadena coincidente anterior. Cada uno de los bytes que forman el mapa de bits, salvo quizá el último, contiene 8 bits y por lo tanto hace referencia a 8 elementos de la estructura de la zona 3. El último byte del mapa de bits contendrá entre 1 y 8 bits significativos (el resto hasta 8 tendrá valor 0).
3. Caracteres y Referencias: comienza con 8 caracteres idénticos a los 8 primeros caracteres del texto original (texto no comprimido). A continuación contiene una ristra de elementos, cada uno de los cuales puede ser un carácter individual (un byte sin signo) o bien un grupo de 3 bytes con los que se especifica la posición y longitud de una subcadena de N caracteres coincidentes con los siguientes a comprimir.

Generación/interpretación del mapa de bits (Comprime/Descomprime)

- El tamaño mínimo de un dato que admiten las lecturas y escrituras en memoria es de 1 byte, 8 bits. Sin embargo, cuando se trata de generar (en 'Comprime') o de interpretar (en 'Descomprime') el mapa de bits, es necesario manejar información a nivel de bit.

Tal como indica el enunciado, para realizar estas operaciones es necesario trabajar con bytes, cada uno de cuyos bits se puede ir rellenando o interpretando sobre la marcha, de modo que tras leer un byte de memoria ('Descomprime') se puedan ir identificando sus bits en el orden indicado en el enunciado. Cuando se complete la preparación de un byte ('Comprime'), este se podrá escribir en memoria.

Para preparar un byte que se va a escribir en memoria, se podrá partir de un registro inicializado a 0 y después ir poniendo a 1 los bits que representen las referencias a subcadenas. Estos bits se podrán poner a 1 con instrucciones lógicas como `or` o aritméticas como `add` o de campo de bit como `set`, siempre utilizando los parámetros adecuados. Por ejemplo, si se quiere poner a 1 el bit3, se puede hacer un `or` con `00001000` o bien un `add` de un valor 8 (equivalente a ese bit3) o un `set` con `W5=1` y `O5=3`, ya sea en la versión de esa instrucción que usa tres registros o la que usa dos registros y un dato inmediato con formato `W5<O5>`.

Por otra parte, una vez leído un byte, se puede ver de distintas formas si un bit en particular tiene valor 1 o valor 0. Por ejemplo, si se quiere comprobar el valor del bit3, se puede hacer un `and` con `00001000` y ver si el resultado es cero o distinto de cero. O hacer un `or` con ese mismo dato (`00001000`) y ver si el byte inicial ha cambiado o permanece igual. También es posible hacerlo con `extu` seleccionando `W5=1` y `O5=3`, ya sea en la versión de esa instrucción que usa tres registros o la que usa dos registros y un dato inmediato con formato `W5<O5>`. No son las únicas posibilidades, se pueden usar otras instrucciones de campo de bit, como `set` o `c1r` para comparar a continuación si el byte ha sufrido algún cambio. O con instrucciones aritméticas de suma/resta y comparación, aunque esta opción requiere hacer las comprobaciones en cierto orden.

Puede tener en cuenta que los valores concretos indicados, tales como `00001000`, pueden especificarse de ese modo, como dato inmediato, o bien como el contenido de un registro.

Para aclarar en lo posible la interpretación del último argumento de las instrucciones de campo de bit, incorporamos aquí una explicación alternativa a la que se facilita en el manual del 88110.

Este último argumento está formado por dos campos de 5 bits cada uno: `o5` (los 5 bits menos significativos, bit4 a bit0) que determinan el **O**rigen (primer bit) del campo a construir y puede valer desde `o5=0` hasta `o5=31`.

Los siguientes 5 bits, `w5` (bit9 a bit5), indican la anchura ("**W**idth") del campo de bits que se construye, anchura que puede valer desde `w5=0` hasta `w5=31`.

Ese tercer argumento se puede definir como un dato inmediato que se expresa de la forma

especial w_5 o bien puede ser un registro, p.e. r_2 , de cuyo contenido se interpretarán los 10 bits menos significativos como w_5 (los bits 9 a 5) y o_5 (los bits 4 a 0).

De este modo, tras ejecutar las dos instrucciones `mak` del siguiente fragmento de código quedaría el mismo resultado en r_{10} y r_{11} :

```
add r20, r0, 0xFFFF
mak r10, r20, 1<3>
add r2, r0, 32 ; bits 5 a 9 de r2=00001 ( $w_5 = 1$ )
add r2, r2, 3 ; bits 0 a 4 de r2=00011 ( $o_5 = 3$ )
mak r11, r20, r2 ; bits 0 a 9 de r2: 0000100011
```

Página actualizada el 24 de noviembre de 2025

[Ir al Principio](#)